

EduGenie: AI-Powered Learning Assistant

Project Description:

EduGenie – AI-Powered Learning Assistant is an educational application designed to support learners through AI-assisted study and learning activities. The solution is organized around five functional features: Ask a Question, Explain a Topic, Generate Quiz, Summary, and Recommended Learning Path. These features are intended to provide learners with explanations, practice, concise study material, and guided learning support.

Core Features:

- **Ask a Question:** helps a learner obtain an AI-generated response to a question.
- **Explain a Topic:** provides an explanation of a selected learning topic.
- **Generate Quiz:** creates quiz-style practice content for learning and revision.
- **Summary:** converts learning content into a concise summary for easier revision.
- **Recommended Learning Path:** provides a structured sequence of suggested learning steps.

Project Details:

Team ID: UNM1659GenieBirds

Team Leader: Priyadharshini G

Team Members: Bushra Fathima T, Pooja M

Scenarios

- A learner asks EduGenie a subject-related question and receives an AI-assisted explanation.
- A learner enters a difficult topic and uses the Explain a Topic feature to understand the concept step by step.
- A learner preparing for an assessment generates a quiz, reviews a summary, and follows a recommended learning path.

Technical Architecture

Description: EduGenie follows a modular application architecture in which the user interface collects learning requests, the application layer processes the request, and an AI service produces the learning response. The five functional features are exposed through the application interface and return structured learning content to the user.

Logical Layers

- **Presentation Layer:** user-facing screens and controls for submitting learning requests and viewing results.
- **Application Layer:** request handling, validation, feature selection, prompt/request construction, and response formatting.
- **AI Integration Layer:** communication with the configured AI service and handling of generated learning content.

Database Note: The supplied sample documentation states that an ER diagram is required when a database is used. No database design details were supplied for EduGenie in the material available for this file, so no ER diagram is claimed here.

Pre-requisites

- Basic programming knowledge and familiarity with the project's chosen development environment.
- Understanding of web/application development concepts.
- Familiarity with AI/API integration and environment-variable based configuration.
- Git/version-control knowledge for obtaining and managing the project source.
- Internet access where the AI service requires an online API connection.

Project Workflow

Milestone 1: Solution Planning and Architecture

- Activity 1.1: Define EduGenie's learning-assistant objectives and target users.
- Activity 1.2: Identify the five functional features included in the solution.
- Activity 1.3: Define the interaction between the user interface, application logic, and AI service.
- Activity 1.4: Prepare the development environment and project dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop Ask a Question, Explain a Topic, Generate Quiz, Summary, and Recommended Learning Path.
- Activity 2.2: Implement request handling and response presentation for each feature.

Milestone 3: Application Logic Development

- Activity 3.1: Implement the main application flow and feature-specific processing.
- Activity 3.2: Add input validation and response handling.

Milestone 4: Frontend Development

- Activity 4.1: Design a simple and learner-friendly interface.
- Activity 4.2: Connect user actions to the learning features and display generated results.

Milestone 5: Testing and Deployment

- Activity 5.1: Run functional and performance checks.
- Activity 5.2: Prepare the project for local or hosted execution.

Milestone 6: Documentation and Conclusion

Document the project structure, features, execution procedure, testing approach, limitations and future improvements.

Milestone 1: Solution Planning and Architecture

This milestone establishes the learning goals and technical structure of EduGenie. The solution is organized around a small set of focused features so that a learner can move from asking a question to practicing and planning further study.

Activity 1.1: Define Project Objectives

- Provide AI-assisted learning support in a single application.
- Make explanations and study support accessible through simple user interactions.
- Support both immediate questions and structured revision activities.

Activity 1.2: Define Functional Features

- Ask a Question
- Explain a Topic
- Generate Quiz
- Summary
- Recommended Learning Path

Activity 1.3: Define Application Architecture

- Identify the presentation layer for learner interaction.
- Identify the application layer for validation and feature processing.
- Identify the AI integration layer for generated responses.

Activity 1.4: Development Environment

Install the software, libraries and dependencies specified by the actual EduGenie source package. Keep secrets such as API keys outside the source code and use environment configuration where required.

Activity 1.1: Project Requirements

Functional Requirements

- The learner should be able to submit a question.
- The learner should be able to request an explanation for a topic.
- The learner should be able to generate quiz content.
- The learner should be able to obtain a summary.
- The learner should be able to request a recommended learning path.

Non-Functional Requirements

- The interface should be understandable and easy to navigate.
- Requests should be validated before processing.
- Generated responses should be displayed clearly.
- The application should handle service or network errors gracefully.
- Sensitive configuration values should not be hard-coded into public source files.

User Flow

Learner opens EduGenie → selects a learning feature → enters the required learning input → submits the request → application validates and processes the request → AI service generates learning content → result is displayed to the learner.

Activity 1.2: Feature Design

Ask a Question

Accepts a learner's question and returns an AI-assisted answer intended to support understanding.

Explain a Topic

Accepts a topic and provides a learner-oriented explanation. The actual prompt structure and output format should follow the implemented project code.

Generate Quiz

Creates practice questions from the learner's selected topic or input, supporting revision and self-checking.

Summary

Produces concise learning content from the supplied material or topic so the learner can revise key points efficiently.

Recommended Learning Path

Suggests an ordered set of learning activities or topics based on the learner's selected subject or learning request.

Activity 1.3: Application Architecture Flow

High-level flow:

Learner → User Interface → Application Request Handler → Feature Logic → AI Service → Response
Validation/Formatting → User Interface

Input Handling

- Read the learner's selected feature and input.
- Validate required fields before sending the request.
- Reject empty or invalid requests with a clear user-facing message.

AI Integration

- Construct a feature-specific request/prompt.
- Send the request through the configured AI integration.
- Receive and validate the generated response.
- Return a structured result to the interface.

Security

Store API credentials or other secrets in environment configuration rather than directly in source code. The exact provider and variable names should match the actual EduGenie implementation.

Milestone 2: Core Functionalities Development

The second milestone implements EduGenie's five functional features as the main learner-facing capabilities.

Activity 2.1: Develop the Core Features

- Implement the question-answering flow.
- Implement topic explanation.
- Implement quiz generation.
- Implement summary generation.
- Implement recommended learning path generation.

Activity 2.2: Common Request Processing

- Receive user input.
- Validate the input.
- Select the appropriate feature logic.
- Build the AI request.
- Process the AI response.
- Display the result in a readable format.

Milestone 3: Application Logic Development

The application logic acts as the connection between the user interface and the AI service.

Activity 3.1: Main Application Flow

- Initialize the application and configuration.
- Register the routes, handlers or feature actions used by the interface.
- Receive user requests.
- Validate and sanitize inputs.
- Call the selected feature logic.
- Return the generated result or an error message.

Activity 3.2: Error Handling

- Handle missing user input.
- Handle unavailable AI/API services.
- Handle malformed or unexpected AI responses.
- Show a useful error message without exposing secret configuration.

Milestone 3: Prompt / Request Design

Each EduGenie feature should use a request structure appropriate to its educational purpose.

Ask a Question

Request the AI service to answer the learner's question clearly and in a learner-friendly manner.

Explain a Topic

Request a structured explanation appropriate to the learner's topic and the application's intended audience.

Generate Quiz

Request practice questions in the format implemented by the application, with clear questions and answers/explanations where supported.

Summary

Request the main ideas from the supplied learning content without unnecessary repetition.

Recommended Learning Path

Request an ordered learning sequence that moves from foundational material toward the requested learning goal.

Response Validation and Presentation

Response Validation

- Check that the AI service returned a response.
- Validate the expected response structure where structured output is used.
- Handle incomplete or malformed output safely.
- Avoid displaying raw service errors to the learner when a clearer application message can be shown.

Presentation

- Keep headings and results visually separated.
- Use readable text and consistent spacing.
- Make generated quiz questions easy to scan.
- Present summaries as concise learning notes.
- Present learning-path steps in an ordered sequence.

Important implementation note: Exact function names, routes, libraries and response schemas should be taken from the submitted EduGenie source code.

Milestone 4: Frontend Development

The frontend should provide a straightforward learning experience in which the learner can select a feature, provide input and view the generated result.

Activity 4.1: User Interface

- Provide clear controls for the five EduGenie features.
- Provide input fields appropriate to the selected feature.
- Provide a visible action control for submitting the request.
- Display loading/progress feedback while an AI response is being generated.
- Display results in a readable and organized format.

Activity 4.2: Dynamic Results

- Update the result area after a successful response.
- Keep previous content from interfering with a new request.
- Show validation or service errors clearly.
- Keep the interface usable across supported screen sizes.

Feature Interface Details

Ask a Question

Input: learner question. Output: AI-assisted answer.

Explain a Topic

Input: topic name or learning prompt. Output: topic explanation.

Generate Quiz

Input: subject/topic and any supported quiz options. Output: generated quiz content.

Summary

Input: learning text/topic. Output: concise summary.

Recommended Learning Path

Input: learning goal/topic. Output: ordered learning suggestions.

The exact fields and controls should match the final EduGenie implementation.

Frontend Interaction Flow

Typical interaction

- Learner selects one of the five features.
- The interface reveals the required input controls.
- Learner enters the learning request.
- The application validates the input.
- The request is sent to the backend/AI integration.
- A loading indicator is shown while processing.
- The generated result is rendered in the result area.
- The learner can start another request.

Usability Considerations

- Use simple labels and instructions.
- Keep important actions easy to locate.
- Provide feedback for invalid or empty input.
- Avoid exposing API keys or internal errors in the interface.

Milestone 5: Testing and Local Execution

Before deployment, the EduGenie application should be executed locally and its five features should be checked.

Functional Verification

- Ask a Question returns a response.
- Explain a Topic returns an explanation.
- Generate Quiz returns quiz content.
- Summary returns concise learning content.
- Recommended Learning Path returns an ordered learning sequence.

Input Validation Checks

- Submit an empty request.
- Submit a normal valid request.
- Submit unusually long or unsupported input where applicable.
- Verify that errors are handled without application failure.

Performance Testing

Phase 6 documents performance testing targets and scenarios. Actual measured values should be entered from the performance-testing tool rather than estimated.

Deployment Preparation

Local setup

- Obtain the complete EduGenie source package.
- Install the dependencies specified by the project.
- Create the required environment configuration.
- Start the application using the command documented in README/setup instructions.

Hosted deployment

If EduGenie is hosted, the final documentation should record the actual deployed URL and hosting provider. Do not insert a placeholder as if it were a live URL.

Secrets

Do not include private API keys or passwords in the documentation or public repository. Use environment variables or the configuration mechanism implemented by the project.

Project Execution and Access

Evaluator workflow

- Open the repository or submitted project package.
- Read the README / setup instructions.
- Install the listed dependencies.
- Configure the required environment variables.
- Start the application.
- Open the local or deployed application URL.
- Test each of the five functional features.

Access Details

The exact repository link, hosted URL, login credentials (if any), and demo-video link were not supplied in the source material used for this document. Add the actual values before final submission.

Known Issues and Limitations

Potential limitations to verify in the implemented project

- AI-generated responses depend on the availability and behavior of the configured AI service.
- Network/API latency can affect the time taken to generate responses.
- The quality of generated educational content can vary with the user input and request design.
- A valid AI-service configuration may be required for the AI features to operate.
- Exact deployment limitations depend on the hosting environment.

Handling

- Provide clear error messages for unavailable services.
- Validate input before sending requests.
- Keep configuration secrets outside source control.
- Document actual limitations observed during testing.

Testing Evidence and Demonstration

Suggested evidence for submission

- Screenshot of the EduGenie home/feature-selection interface.
- Screenshot showing a successful Ask a Question response.
- Screenshot showing a generated quiz or summary.
- Screenshot showing a recommended learning path.
- Performance-testing screenshot showing actual response-time/throughput/error results, if performed.

Demo walkthrough

A short demonstration can show: opening EduGenie → selecting a feature → entering a learning request → generating the response → viewing the result → trying another feature.

Only screenshots and measurements actually produced by the team should be inserted as evidence.

Future Enhancements

Possible future improvements

- Personalized learning based on learner progress.
- Persistent learning history and saved results.
- User authentication and individual learner profiles.
- Additional subjects, languages and learning levels.
- More detailed quiz analytics and progress tracking.
- Multimedia learning support where appropriate.
- Improved accessibility and responsive design.
- Additional AI safety and content-quality checks.

These are future enhancement ideas and are not claims that the current EduGenie implementation already contains them.

Conclusion

EduGenie – AI-Powered Learning Assistant is organized as a focused educational solution with five functional features: Ask a Question, Explain a Topic, Generate Quiz, Summary, and Recommended Learning Path. The project documentation describes the intended application flow, feature responsibilities, architecture, testing approach, execution process, limitations and possible future improvements.

For final submission, the documentation should be kept consistent with the actual EduGenie source code. Technology names, routes, dependency files, deployment URLs, repository links and measured test results should be replaced with the project's real values wherever placeholders or general descriptions remain.

Project: EduGenie – AI-Powered Learning Assistant

Team ID: UNM1659GenieBirds

Team: Priyadharshini G, Bushra Fathima T, Pooja M

Documentation complete.